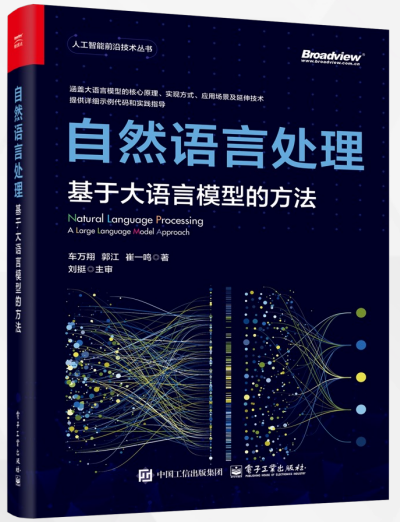


基础工具集与常用数据集

车万翔

社会计算与交互机器人研究中心（HIT-SCIR）
哈尔滨工业大学

工具在自然语言处理实验中的分工

工具	输入与输出	本讲示例
tiktoken	字符串与Token ID相互转换	“我爱北京天安门！”
NLTK / LTP	文本及语言学分析结果	词性、词义、中文分词
PyTorch	张量、计算与可训练参数	两个特征的简单分类器
Datasets / Evaluate	数据加载与指标计算	影评样本、准确率

tiktoken：编码与完整解码

```
import tiktoken
enc = tiktoken.get_encoding("cl100k_base")
text = "Hello tiktoken!"
ids = enc.encode(text)
print(ids)
assert enc.decode(ids) == text
print([enc.decode_single_token_bytes(id) for id in ids])
```

Token ID	9906	87272	5963	0
字节片段对应文本	Hello	tik	token	!

课堂练习：为什么“爱”消失了

```
text = "我爱北京天安门！"
ids = enc.encode(text)
parts = [enc.decode_single_token_bytes(id) for id in ids]
bad = "".join(p.decode("utf-8", errors="ignore")
              for p in parts)
```

操作	本例结果
逐片段解码，忽略错误	我北京天安门!
enc.decode(ids)	我爱北京天安门!
b"".join(parts).decode("utf-8")	我爱北京天安门!

中文Token与UTF-8字节

“我爱北京天安门！”共有8个Token

原文片段	我	爱：前2字节	爱：后1字节	北京
Token ID	37046	76207	109	70090
UTF-8十六进制	E6 88 91	E7 88	B1	E5 8C 97 E4 BA AC

一个Token可能只含一个字的部分字节，也可能覆盖多个字。

后4个Token依次为“天” “安” “门” “！”。Token数量由词表和文本共同决定。

NLTK：英文文本分析与语言资源

语料库 小说文本、带标签的影评
Penn Treebank样例

词典 WordNet词义与语义关系
SentiWordNet情感词典

分析工具 分句、词元解析、词性标注
组块、实体与句法分析



NLTK：分句、词元解析与词性标注

```
import nltk
from nltk import sent_tokenize, word_tokenize, pos_tag
nltk.download("punkt_tab")
nltk.download("averaged_perceptron_tagger_eng")
text = "They sat by the fire. They fire a gun."
sents = sent_tokenize(text)
print([pos_tag(word_tokenize(s)) for s in sents])
```

语境	fire的作用	标签解释
by the fire	名词：火	NN
They fire a gun	动词：开火	VBP

WordNet：词与词义的区别

bank可以表示“河岸”，也可以表示“银行”

```
import nltk
nltk.download("wordnet")
from nltk.corpus import wordnet as wn
for syn in wn.synsets("bank", pos=wn.NOUN):
    print(syn.name(), syn.definition())
```

synset　　把同一概念下的同义词组织在一起

语义关系　　上位词、下位词等关系连接不同概念

停用词处理需要结合任务

文本	删除not后的文本	信息变化
The movie is not good.	The movie is good.	否定信息丢失

```
nltk.download("stopwords")
from nltk.corpus import stopwords
stop = set(stopwords.words("english")) - {"not", "no", "nor"}
words = word_tokenize("The movie is not good.")
kept = [w for w in words if w.lower() not in stop]
print(kept)
```

LTP：中文基础分析平台

词法　　分词、词性、命名实体

句法　　依存句法分析

语义　　语义角色、语义依存



LTP调用示例：Pipeline接口

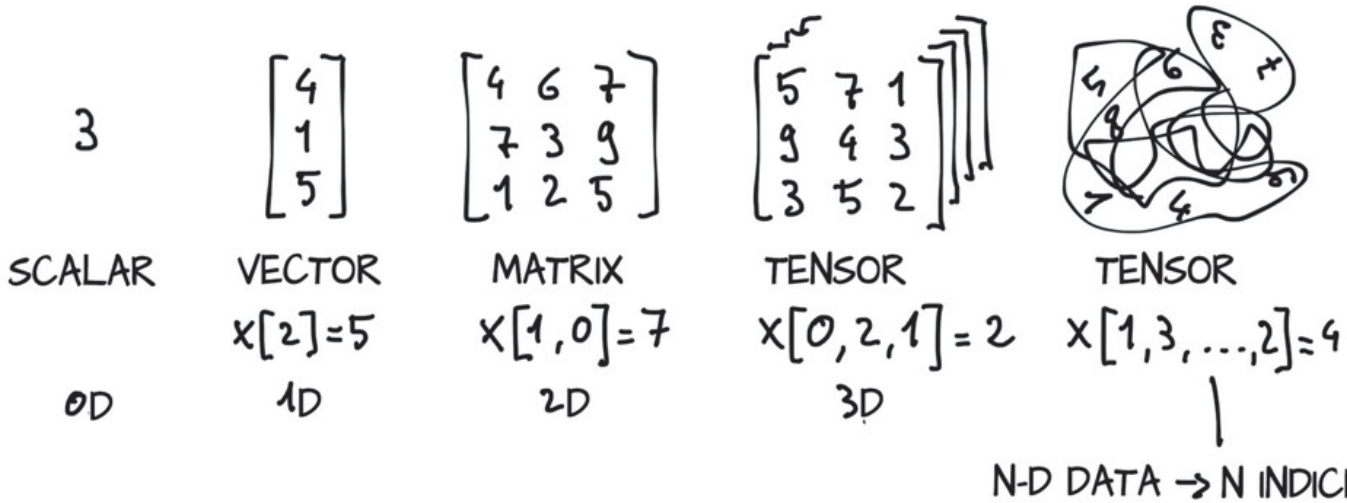
```
from ltp import LTP
ltp = LTP("LTP/small")
out = ltp.pipeline(
    ["南京市长江大桥。"], tasks=["cws", "pos"]
)
print(out.cws)
print(out.pos)
```

教材分词示例	南京市	长江	大桥	。
词性标签示例	ns	ns	n	wp

PyTorch：张量、运算与模块

概念	作用	示例
张量 Tensor	保存数据和参数	词编号、嵌入矩阵
运算与自动微分	计算输出，并求梯度	矩阵乘法、backward()
模块 nn.Module	组织可复用的模型结构	Embedding、Linear

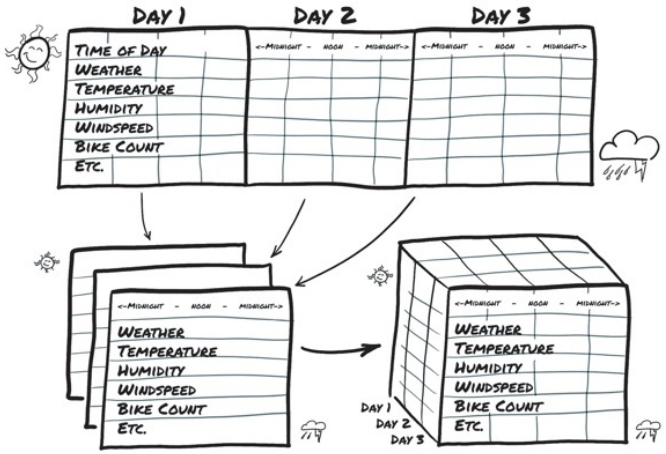
张量：具有形状的多维数组



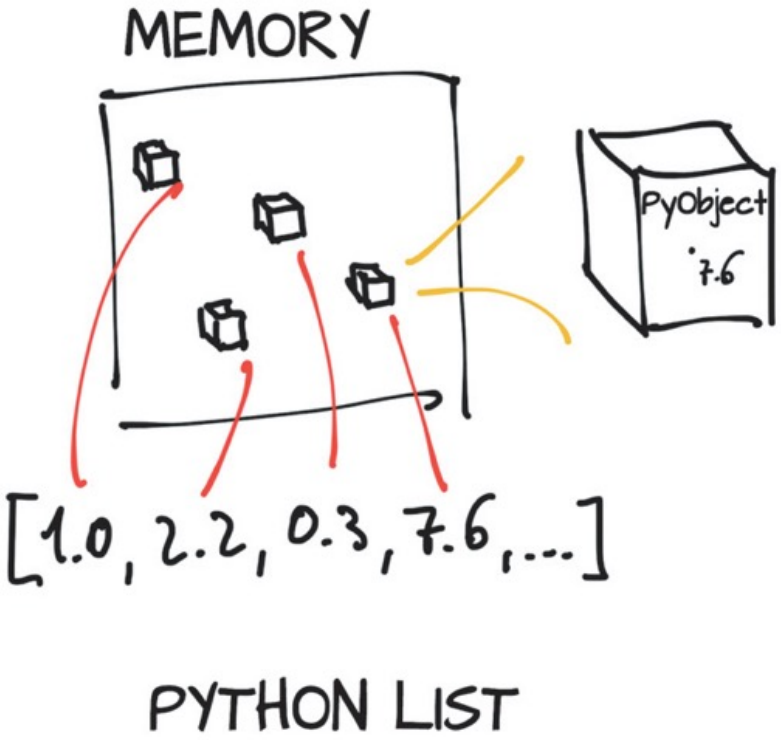
标量　　形状()，例如一个损失值

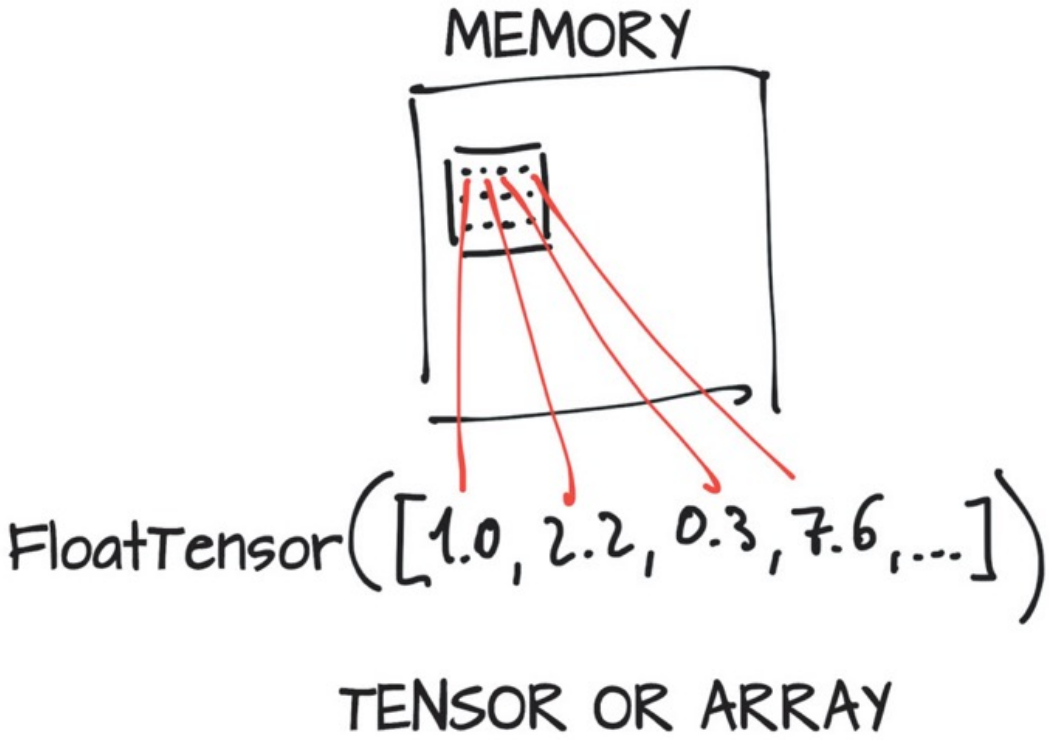
向量　　形状(3,)，例如3个特征

矩阵　　形状(2, 3)，例如2个样本



Python列表与张量存储



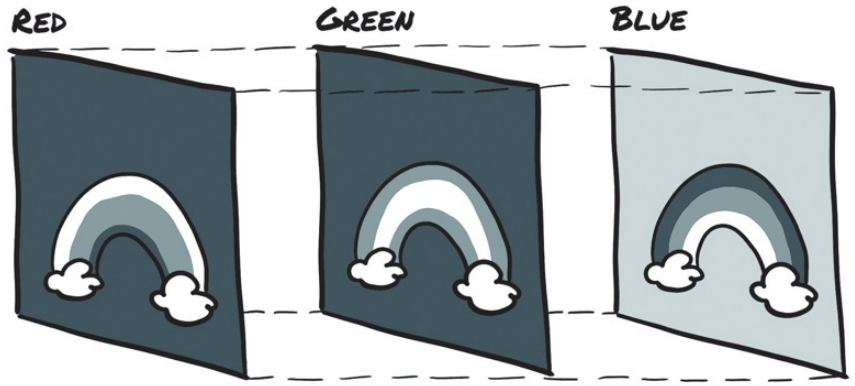


列表：元素可以是不同类型对象

张量：统一dtype，记录shape和stride

NLP中的张量形状与数据类型

对象	形状	常见类型
Token ID	[B, T]	torch.long
词向量	[B, T, D]	浮点数
句子表示	[B, D]	浮点数
分类得分	[B, C]	浮点数



B：批大小　T：序列长度
D：向量维度　C：类别数

张量创建与乘法

```
import torch
x = torch.tensor([1., 2., 3.])
y = torch.tensor([4., 5., 6.])
```

操作	结果	含义
x * y	[4., 10., 18.]	逐元素乘法
x @ y	32.	向量点积
torch.zeros(2, 3)	2行3列，全为0	初始化张量
A @ B	(2,3) @ (3,4) 得到 (2,4)	内侧维度必须匹配

dim与keepdim：沿哪一个轴聚合

```
x = torch.tensor([[1., 2., 3.],
                  [4., 5., 6.]])
```

表达式	数值	形状
x.mean()	3.5	()
x.mean(dim=0)	[2.5, 3.5, 4.5]	(3,)
x.mean(dim=1)	[2., 5.]	(2,)
x.mean(dim=1, keepdim=True)	[[2.], [5.]]	(2, 1)

形状调整：reshape与transpose

x = torch.tensor([[1, 2, 3], [4, 5, 6]])

操作	结果	用途
x.reshape(3, 2)	[[1, 2], [3, 4], [5, 6]]	按顺序重新分组
x.transpose(0, 1)	[[1, 4], [2, 5], [3, 6]]	交换两个轴
x.unsqueeze(0)	形状为(1, 2, 3)	新增长度为1的轴
x.unsqueeze(0).squeeze(0)	形状恢复为(2, 3)	移除指定的单例轴

索引、切片与拼接

x = torch.arange(12).reshape(3, 4)

表达式	结果或形状
x[1, 3]	7
x[:, 2]	[2, 6, 10]
x[:, 2:4]	[[2,3], [6,7], [10,11]]
torch.cat([x, x], dim=0)	形状(6, 4)
torch.cat([x, x], dim=1)	形状(3, 8)

广播：不同形状怎样逐元素运算

```
x = torch.tensor([[1], [2], [3]])  # (3, 1)
y = torch.tensor([[4, 5]])         # (1, 2)
print(x + y)
```

x的值	加4	加5
1	5	6
2	6	7
3	7	8

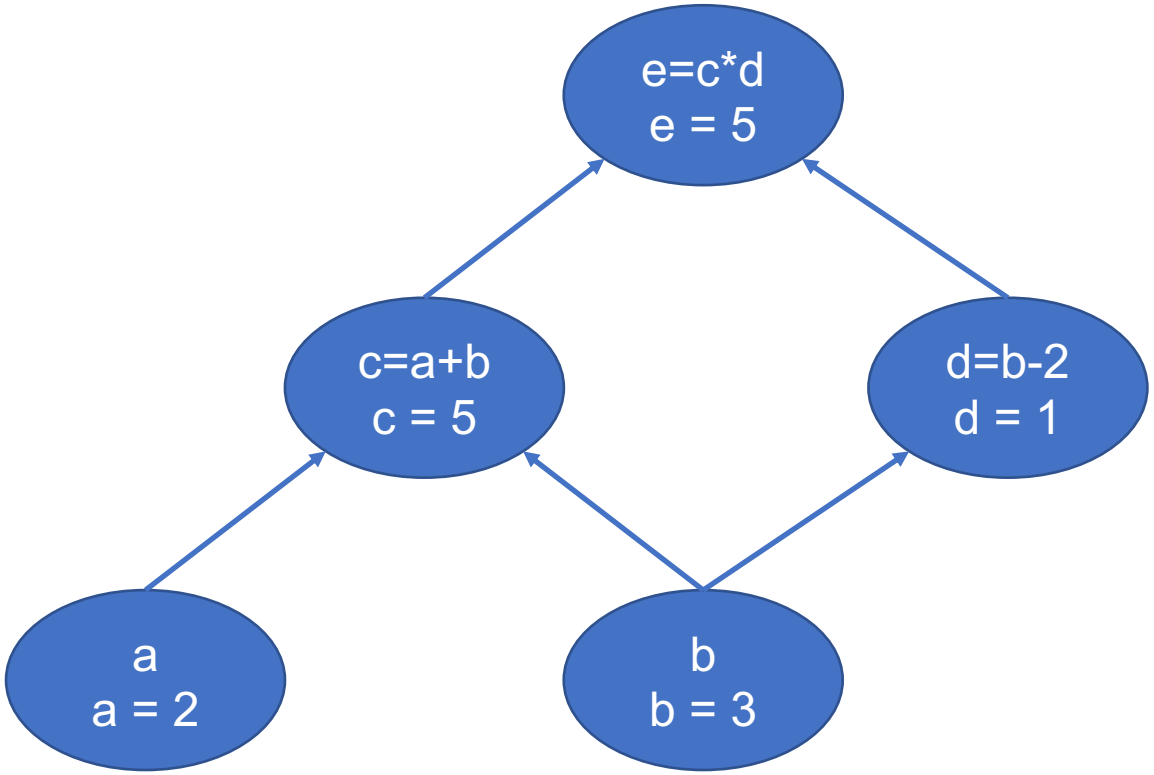
对齐 从最后一个轴开始

兼容 长度相等，或一方为1

输出 形状为(3, 2)

$$e = (a + b) \times (b - 2)$$

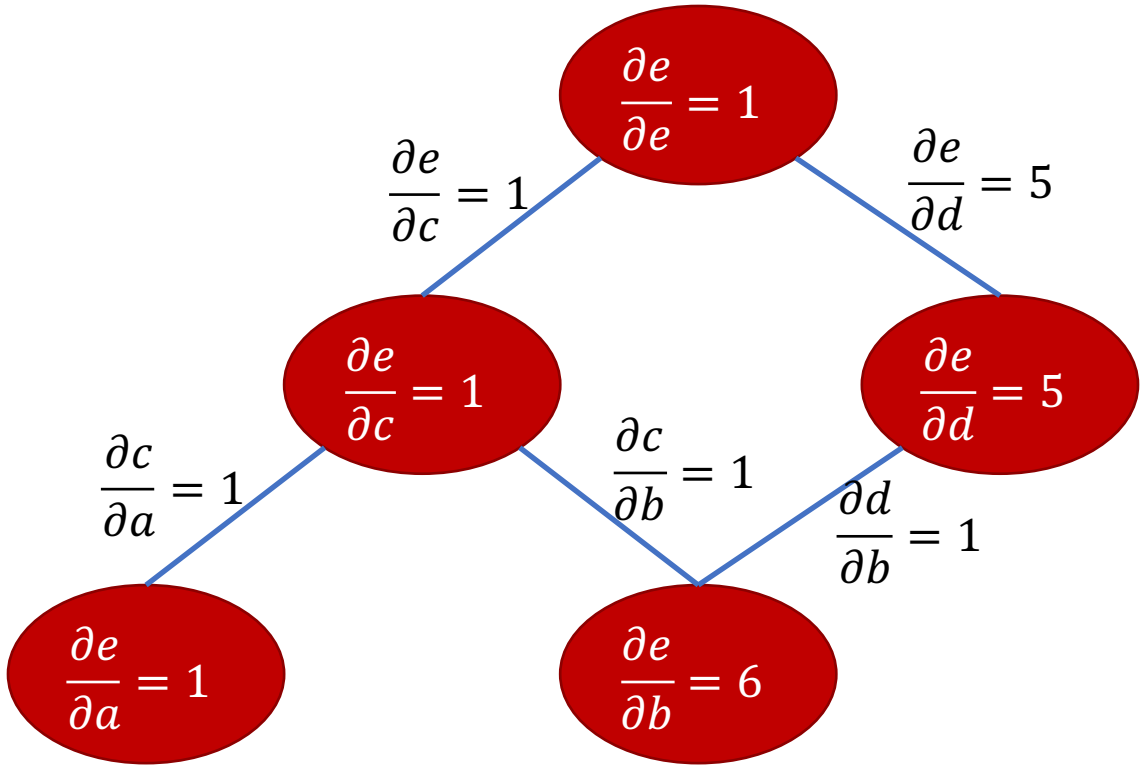
```
a = torch.tensor(2.,
                 requires_grad=True)
b = torch.tensor(3.,
                 requires_grad=True)
c = a + b
d = b - 2
e = c * d
```



```
e.backward()
print(a.grad, b.grad)
# tensor(1.) tensor(6.)
```

∂e/∂a = b − 2 = 1

∂e/∂b = (b − 2) + (a + b)
= 1 + 5 = 6



nn.Module与一个简单分类器

组成	本例对象
torch.nn	nn.Linear(2, 2)
torch.optim	SGD：更新参数
torch.utils.data	Dataset / DataLoader

```
from torch import nn
model = nn.Linear(2, 2)
x = torch.tensor(
    [[1., 0.], [0., 1.]])
logits = model(x)
print(logits.shape)
# torch.Size([2, 2])
```

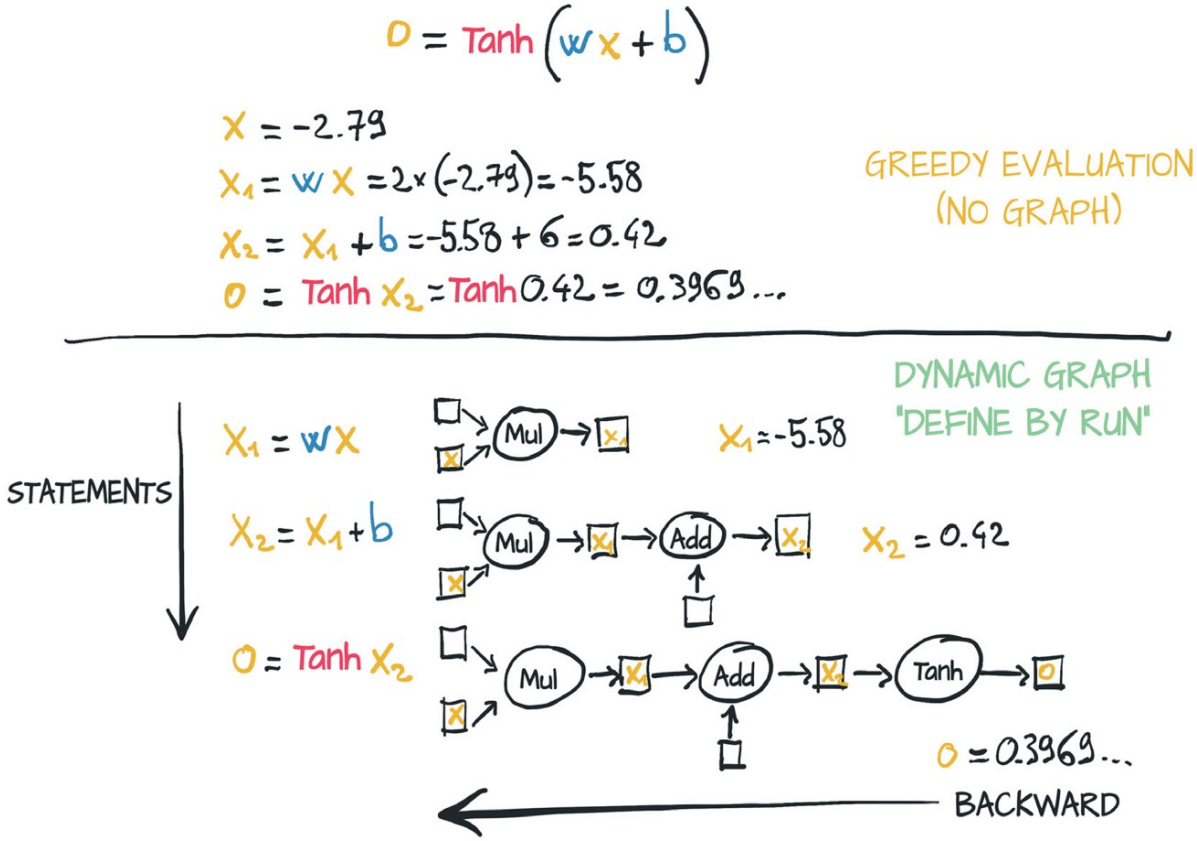
一次参数更新

```
y = torch.tensor([0, 1], dtype=torch.long)
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
loss_fn = nn.CrossEntropyLoss()

optimizer.zero_grad()
logits = model(x)
loss = loss_fn(logits, y)
loss.backward()
optimizer.step()
```

backward()计算梯度，step()根据梯度更新参数。

动态图、设备与编译优化



动态图 便于逐步调试
随执行记录运算

GPU 大批量计算可能加速
数据与模型需在同一设备

编译 torch.compile(model)
可优化部分计算

课堂练习：先预测形状，再解释结果

```
x = torch.tensor([[1., 2., 3.], [4., 5., 6.]])
a = x.mean(dim=1)
b = x.mean(dim=1, keepdim=True)
c = x - b
```

问题	思考重点
a、b、c的形状分别是什么?	哪个轴消失，哪个轴保留?
c中的数值是什么?	每行减去该行的均值
x - a能否直接计算?	从最后一个轴检查广播

预训练语料与任务数据集

用途	样本形式	典型来源
预训练	连续文本，由文本构造预测目标	Wikipedia、Common Crawl
任务训练	输入文本及标签或参考输出	情感分类、NER、问答
评价基准	固定划分、指标和评价协议	SST-2情感任务 CLUE评测任务集合

同样叫“数据集”，训练方式和可用标签可能不同。

维基百科：内容快照与当前下载入口

使用XML快照，复现实验需固定日期

下载内容	包含什么	处理时关注
Content Current	页面当前内容的导出	语言、命名空间、正文
Content History	包含历史版本的内容导出	避免历史修订重复进入训练
SQL表导出	页面元数据或链接关系等	不是纯文本正文语料

2026年官方已推荐MediaWiki Content File Exports

维基百科文本处理示例

阶段	教材例子中的处理
原始内容	<doc id="13" title="数学"> … </doc>
正文抽取	去掉Wiki标记，保留段落和文档边界
按需繁简转换	“數量、結構”转换为“数量、结构”
清洗与保存	处理乱码和残留标签，保留ID与来源

```
with open("sample.txt", encoding="utf-8") as f:
    for line in f:
        text = line.rstrip("\n")
```

Common Crawl：网页数据的不同层次

文件类型	主要内容	适合做什么
WARC	抓取的原始响应与内容	重新进行网页解析
WAT	提取的元数据	研究链接和网页属性
WET	抽取的文本内容	进一步做语言筛选与清洗

常见噪声　　广告、导航、重复文本、编码错误

处理工具　　教材CCNet，及可复现的数据处理流水线

教材中的代表性预训练语料

资源	教材中的特点	学习重点
C4 / mC4 / CC-100	从Common Crawl派生	不同过滤和语言配置
BookCorpus / The Pile	书籍或多来源英文语料	领域、来源与访问条件
ROOTS	BLOOM使用的多语言语料	语言和领域的混合
RedPajama / SlimPajama	公开数据及清洗去重版本	处理前后数据差异
悟道 / CLUECorpus2020 / MNBVC	中文预训练资源	领域覆盖与数据说明

数据清洗：哪些内容应该保留

教学片段	问题	处理思路
首页 注册 登录 首页 注册	网页模板噪声	正文抽取、模板识别
“模型需要数据。” 出现100次	重复样本	精确与近重复检测
if x < 3: print(x)	有效代码	根据目标保留代码
“这个结果并不差。”	包含否定词	保留否定与语境
同一新闻的原文与转载	来源重复	分组去重后划分数据

删除规则越强，是否一定得到更好的训练数据?

研究进展：数据筛选与多语言处理

代表工作	方法	对基础工具课的启发
FineWeb-Edu（2024）	模型辅助标注质量 小分类器筛选网页	筛选目标会影响 最终学到的能力
FineWeb2（2025论文）	面向多语言调整 清洗和去重流程	同一规则在不同语言 上可能产生不同效果

具体例子：全角中文标点是否应该一律变成半角？

Hugging Face Datasets：加载与检查

```
from datasets import load_dataset
ds = load_dataset("stanfordnlp/sst2",
                  split="validation[:5]")
print(ds.features)
print(ds[0])
```

字段	含义
sentence	英文影评文本
label	0表示负面，1表示正面
idx	样本编号

本地数据与批量预处理

```
from datasets import Dataset
ds = Dataset.from_dict({
    "text": ["很满意", "非常差"],
    "label": [1, 0]
})
ds = ds.map(lambda row: {"n_chars": len(row["text"])})
print(ds.column_names)
```

功能	常见操作
加载本地文件	load_dataset("json", data_files="train.jsonl")
预处理	map增加字段，filter筛选样本
框架格式	with_format("torch")转换数值字段

流式读取：先处理一个小样本

```
stream = load_dataset(
    "HuggingFaceFW/fineweb-2",
    name="hrv_Latn", split="train", streaming=True
)
for row in stream.take(3):
    print(row["text"][:120])
```

按需读取　　无需先把整个数据集下载到本地

顺序迭代　　访问方式与普通可随机索引的数据集不同

训练、验证与测试：数据泄漏

划分	用途	常见误用
训练集	学习模型参数	把测试样本混入训练
验证集	选择模型和超参数	把验证结果当最终无偏估计
测试集	最终评估泛化能力	反复看分数，继续调参

案例：同一篇新闻的原文在训练集，转载在测试集。

按来源分组、检查近重复，必要时按时间划分。

Evaluate：指标计算与解释

```
import evaluate
metric = evaluate.load("accuracy")
result = metric.compute(
    predictions=[1, 1, 0],
    references=[0, 1, 0]
)
print(result)  # {"accuracy": 0.6666666666666666}
```

样本	1	2	3
是否预测正确	否	是	是

课堂小结与课后实验

实验任务	提交内容
中文Token检查	原文、Token ID、字节片段、往返断言
张量与自动微分	关键shape、手算结果、运行输出
小数据加载与评价	字段说明、数据划分、指标及错误案例

每个实验同时记录：代码、依赖版本、数据或模型版本。

教材与延伸阅读

教材	第3章：基础工具集与常用数据集
文本工具	github.com/openai/tiktoken www.nltk.org　　github.com/HIT-SCIR/ltp
深度学习	docs.pytorch.org/tutorials/beginner/basics/intro.html
数据与指标	huggingface.co/docs/datasets huggingface.co/docs/evaluate
研究选读	FineWeb数据集（2024） FineWeb2论文：arxiv.org/abs/2506.20920

谢 谢!

理解语言，认知社会

以中文技术，助民族复兴



长按二维码，关注哈工大SCIR

微信号：HIT_SCIR